AES.c

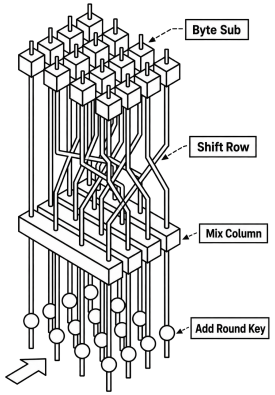
ADVANCED ENCRYPTION STANDARD

(RIJNDAEL)

Design, implementation and modes of operation

```
$ ./AES e 1 in.pdf out.pdf KEY
> key expansion ..... ok
> rounds ..... 10
```

AES round



ARGHA SARDAR

IISc // October 14, 2026

AGENDA

```
> ls talk/
```

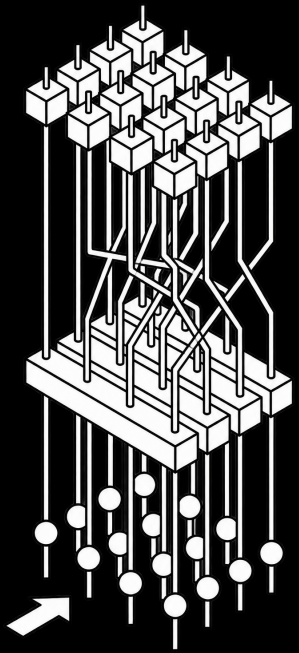
01 introduction: block ciphers, history, parameters
02 preliminaries: the state, arithmetic in $\mathrm{GF}(2^8)$, the S-box
03 encryption: SubBytes, ShiftRows, MixColumns, AddRoundKey, KeyExpansion
04 decryption: InvCipher and the equivalent inverse cipher
05 look-up table implementation (T-tables)
06 modes of operation: ECB, CBC, OFB, CTR
07 our C implementation and references

Notation follows FIPS 197: state $s_{r,c}$, round count N_r, key words N_k, $N_b = 4$, transformed values are primed (s'), $\oplus$ is XOR, $\bullet$ is multiplication in $\mathrm{GF}(2^8)$.

> cd section_01

01

INTRODUCTION



> A **block** is a bit sequence of fixed length n.

> A **block cipher** is a map

$$E: \ \mathcal{K} \times \{0,1\}^n \to \{0,1\}^n,$$

such that for every $K \in \mathcal{K}$ the function $E_K(\cdot) \triangleq E(K,\cdot)$ is a **bijection** of $\{0,1\}^n$.

> Decryption is the inverse permutation: $D_K = E_K^{-1}$.

> A block cipher is a fundamental primitive and the basis of services that provide **confidentiality** (via modes of operation, Sec. 06).

AES as a block cipher

$n = 128$ bits (16 bytes)

$|K| \in \{128, 192, 256\}$ bits

$E_K =$ Cipher$(\cdot, N_r, \text{KeyExpansion}(K))$

Cipher is a permutation of $\{0,1\}^{128}$ for each key; InvCipher is its inverse.

[Rijndael AES Implementation §1; FIPS 197 §3.1, §5]

HISTORY OF AES (1/2): THE COMPETITION

2 Jan 1997	NIST announces the search for an unclassified, publicly disclosed successor to DES (published 1977).
12 Sep 1997	Formal call: block ciphers with 128-bit blocks and 128/192/256-bit keys.
1998	15 designs submitted (incl. MARS, RC6, Rijndael, Serpent, Twofish); conferences AES1 (Aug 1998), AES2 (Mar 1999).
Aug 1999	Five finalists: **MARS, RC6, Rijndael, Serpent, Twofish**.
3 Sep 1999	Daemen and Rijmen (Belgium) submit "AES Proposal: Rijndael", derived from the Square cipher.
Apr 2000	AES3 conference: finalist teams present, then a vote.
2 Oct 2000	NIST announces **Rijndael** as the proposed AES.

[Wikipedia: "Advanced Encryption Standard" and "AES process"; Rijndael AES Implementation §1]

HISTORY OF AES (2/2): STANDARD AND CRYPTANALYSIS

26 Nov 2001	Published as **FIPS PUB 197**; effective as a US federal standard 26 May 2002; later ISO/IEC 18033-3.
Jun 2003	Approved for US government information up to SECRET (all key sizes); TOP SECRET needs 192/256-bit keys.
2005-2010	Cache-timing attacks on table-based software (Bernstein 2005; Osvik-Shamir-Tromer); Intel AES-NI hardware instructions (2010).
2009-2011	Related-key attacks on AES-192/256 (not on the full cipher as used); first key recovery on full AES by biclique attack (Bogdanov-Khovratovich-Rechberger 2011): $2^{126.2}$ for AES-128, about $4\times$ faster than brute force.
2015	Tao-Wu improve it to $2^{126.0}$, $2^{189.9}$, $2^{254.3}$ (AES-128/192/256).
9 May 2023	FIPS 197 updated (update 1).

No practical attack is known against correctly implemented AES.

[Wikipedia: "Advanced Encryption Standard"; FIPS 197; Rijndael AES Implementation §5.1, §5.4]

AES PARAMETERS

	N_k	key bits	N_b	N_r
AES-128	4	128	4	10
AES-192	6	192	4	12
AES-256	8	256	4	14

$$\text{AES-}k(in, key) = \text{Cipher}(in, N_r, \text{KeyExpansion}(key))$$

- Key schedule: $4(N_r + 1)$ words $w[i]$.
- AES-128 needs $4 \cdot 11 = 44$ words $= 176$ bytes.
- Our implementation: **AES-128 only**.

utils.h

```
#define NK 4  // Number of 32-bit words
              // in the key (AES-128: Nk=4)
#define NR 10 // Number of rounds
              // (AES-128: Nr=10)
typedef unsigned char u8;
void KeyExpansion(u8* key, u8* w);
void Cipher(u8* state, u8* ExpandedKey);
...
void InvCipher( u8* state, u8* ExpandedKey);
```

AES.c

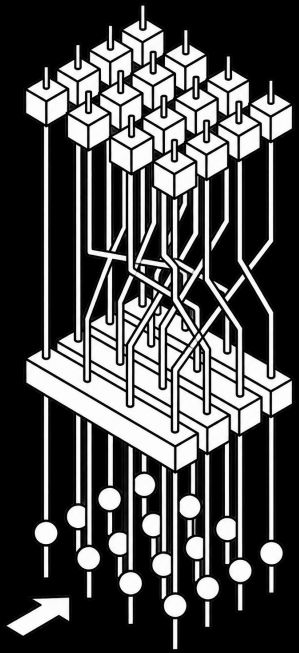
```
u8 key[16];
...
u8 ExpandedKey[176];
KeyExpansion(key, ExpandedKey);
```

[FIPS 197 §5, Table 3; AESFINAL/utils.h, AES.c]

> cd section_02

02

PRELIMINARIES



> Input block $in_0, \ldots, in_{15}$ (bytes) is copied into the **state** s, a 4×4 byte array, **column by column**:

$$s[r, c] = in[r + 4c], \qquad out[r + 4c] = s[r, c],$$

for $0 \leq r < 4$, $0 \leq c < 4$.

> A **word** is 4 bytes. Column c of the state is the word $v_c = (s_{0,c}, s_{1,c}, s_{2,c}, s_{3,c})^T$.

> The key schedule is an array $w[0..4(N_r+1)-1]$ of words; $w[i..i+3]$ denotes four consecutive words.

$s_{0,0}$	$s_{0,1}$	$s_{0,2}$	$s_{0,3}$
$s_{1,0}$	$s_{1,1}$	$s_{1,2}$	$s_{1,3}$
$s_{2,0}$	$s_{2,1}$	$s_{2,2}$	$s_{2,3}$
$s_{3,0}$	$s_{3,1}$	$s_{3,2}$	$s_{3,3}$

$=$

in_0	in_4	in_8	in_{12}
in_1	in_5	in_9	in_{13}
in_2	in_6	in_{10}	in_{14}
in_3	in_7	in_{11}	in_{15}

utils.h

```
typedef unsigned char u8;
void KeyExpansion(u8* key, u8* w);
void Cipher(u8* state, u8* ExpandedKey);
```

u8 state[16] holds the state in **column-major** order: $s_{r,c} = \texttt{state}[\, r + 4c \,]$.

Column c is the contiguous slice state[4c..4c+3], which is why MixColumns works on s[0..3], s[4..7], s[8..11], s[12..15].

Row r is strided: state[r], state[r+4], state[r+8], state[r+12].

[FIPS 197 §3.4-3.5, Fig. 1; AESFINAL/utils.h]

ARITHMETIC IN $\mathrm{GF}(2^8)$: ADDITION AND MULTIPLICATION

byte ↔ polynomial

$\{b_7 b_6 \dots b_0\} \quad \leftrightarrow \quad b(x) = \sum_{i=0}^{7} b_i x^i, \quad b_i \in \mathbb{F}_2$

addition = XOR

Coefficients are added mod 2, so $a + b = a \oplus b$; addition equals subtraction and $a \oplus a = \{00\}$.

$\{57\} \oplus \{83\} = \{d4\}$

$(x^6+x^4+x^2+x+1) + (x^7+x+1) = x^7+x^6+x^4+x^2$

multiplication

$$b \bullet c \;=\; b(x)\, c(x) \bmod m(x)$$

with the fixed irreducible polynomial

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\{11b\}).$$

inverse

For $b \neq \{00\}$ there is a unique b^{-1} with $b \bullet b^{-1} = \{01\}$:

$$b^{-1} = b^{254}.$$

(or via extended Euclid: $b(x)a(x) + m(x)c(x) = 1$).

[FIPS 197 §4.1-4.4; Rijndael AES Implementation §2.1]

XTIME: MULTIPLICATION BY $\{02\}$

Multiplying by x shifts the polynomial; reduce by $m(x)$ if the degree reaches 8:

$$\text{xtime}(b) = b \bullet \{02\} = \begin{cases} \{b_6 b_5 \dots b_0\ 0\} & b_7 = 0 \\ \{b_6 b_5 \dots b_0\ 0\} \oplus \{1b\} & b_7 = 1 \end{cases}$$

Any constant is a sum of powers of $\{02\}$; with $b = \{57\}$:

$\{57\} \bullet \{02\} = \{ae\},\ \{04\}: \{47\},\ \{08\}: \{8e\},\ \{10\}: \{07\}$

$\{13\} = \{10\} \oplus \{02\} \oplus \{01\}$, hence

$\{57\} \bullet \{13\} = \{57\} \oplus \{ae\} \oplus \{07\} = \{fe\}$

Horner form for $\{15\} = \{01\} \oplus \{04\} \oplus \{10\}$:

$b \bullet \{15\} = b \oplus \text{xtime}^2\big(b \oplus \text{xtime}^2(b)\big)$

utils.c :: xtime (constant time)

```
// xtime function: multiplies by
// 0x02 in GF(2^8)
u8 xtime(u8 b) {
    return (b << 1) ^ ((b >> 7) * 0x1B);
}
```

Branching version, **data-dependent timing**:
`(b<<1) ^ ((b & 0x80) ? 0x1B : 0x00)`

Masking version: $b_7 \in \{0, 1\}$ times $\{1b\}$ gives the same value with **no branch**, so the cycle count is independent of b.

[FIPS 197 §4.2 (Eq. 4.5-4.7); Rijndael AES Implementation §2.1; AESFINAL/utils.c]

THE S-BOX AS $f \circ g$: INVERSION AND AFFINE MAP

$$\mathrm{SBox}(b) = f\big(g(b)\big)$$

1. Inversion in $\mathrm{GF}(2^8)$:

$$g(b) = \tilde{b} = \begin{cases} \{00\} & b = \{00\} \\ b^{-1} & b \neq \{00\} \end{cases}$$

2. Affine map, $c = \{63\}$, indices mod 8:

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \tilde{b}_0 \\ \tilde{b}_1 \\ \tilde{b}_2 \\ \tilde{b}_3 \\ \tilde{b}_4 \\ \tilde{b}_5 \\ \tilde{b}_6 \\ \tilde{b}_7 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

$$b'_i = \tilde{b}_i \oplus \tilde{b}_{i+4} \oplus \tilde{b}_{i+5} \oplus \tilde{b}_{i+6} \oplus \tilde{b}_{i+7} \oplus c_i$$

> g is non-linear (an involution, $g \circ g = \mathrm{id}$); f is a bijection; so $\mathrm{SBox}^{-1} = g \circ f^{-1}$.

> Examples: $\mathrm{SBox}(\{53\}) = \{ed\}$, $\mathrm{SBox}(\{00\}) = \{63\}$.

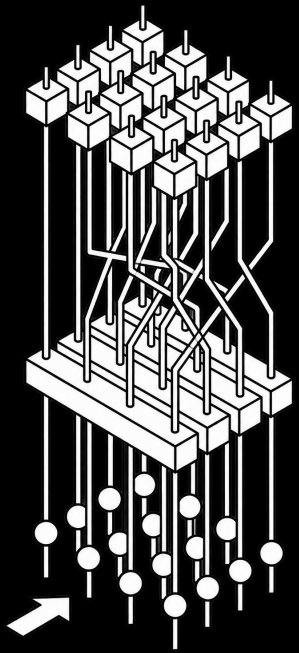
> We use a **256-byte look-up table** for speed (Rijndael AES Implementation §2.2).

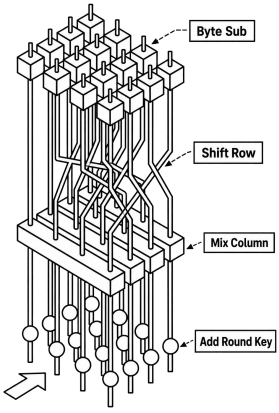
[FIPS 197 §5.1.1 (Eq. 5.2-5.4); Rijndael AES Implementation §2.2]

> cd section_03

03

ENCRYPTION





Each round applies four byte-oriented transformations to the state:

1. SubBytes: S-box applied to each of the 16 bytes (non-linear).
2. ShiftRows: row r is rotated left by r positions.
3. MixColumns: each column times a fixed matrix over $\mathrm{GF}(2^8)$.
4. AddRoundKey: XOR with a 128-bit round key from KeyExpansion.

structure (round index: steps)

0:	AddRoundKey
$1..N_r-1$:	SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey
N_r:	SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey

$N_r + 1$ calls of AddRoundKey; the final round omits MixColumns.

[FIPS 197 §5.1; figure: Byte Sub / Shift Row / Mix Column / Add Round Key (AES_FRONT.png)]

```
1  procedure Cipher(in, N_r, w)
2      state ← in
3      state ← AddRoundKey(state, w[0..3])
4      for round = 1 to N_r − 1 do
5          state ← SubBytes(state)
6          state ← ShiftRows(state)
7          state ← MixColumns(state)
8          state ← AddRoundKey(state,
               w[4 round..4 round+3])
9      end for
10     state ← SubBytes(state)
11     state ← ShiftRows(state)
12     state ← AddRoundKey(state, w[4N_r..4N_r+3])
13     return state
```

utils.c :: Cipher

```
void Cipher(u8* state, u8* ExpandedKey) {
    // Initial round key addition.
    AddRoundKey(state, ExpandedKey, 0);

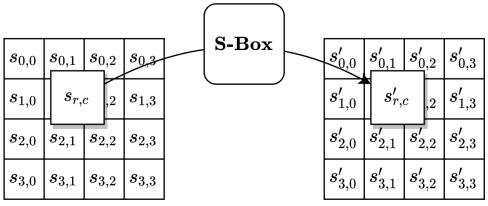
    // 9 main rounds.
    for (u8 round = 1; round < NR; ++round) {
        SubBytes(state);
        ShiftRows(state);
        MixColumns(state);
        AddRoundKey(state, ExpandedKey, round);
    }

    // Final round (no MixColumns).
    SubBytes(state);
    ShiftRows(state);
    AddRoundKey(state, ExpandedKey, 10);
}
```

[FIPS 197 Alg. 1; AESFINAL/utils.c]

Non-linear, invertible, applied **independently** to every byte:

$$s'_{r,c} = \mathrm{SBox}(s_{r,c}), \qquad 0 \le r, c < 4$$



utils.c :: sbox[256], SubBytes

```
const u8 sbox[256] = {
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b,
    0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b,
    ...
};

void SubBytes(u8* state) {
    state[0]  = sbox[state[0]];
    state[1]  = sbox[state[1]];
    state[2]  = sbox[state[2]];
    ...
    state[14] = sbox[state[14]];
    state[15] = sbox[state[15]];
}
```

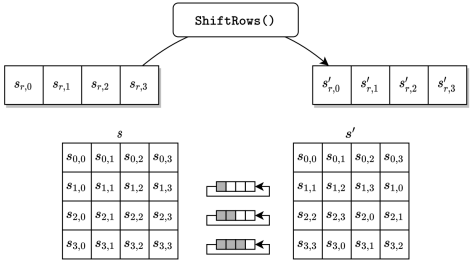
> Table: $\mathrm{SBox}(\{xy\})$ is the entry in row x, column y. E.g. $s_{r,c} = \{53\} \Rightarrow s'_{r,c} = \{ed\}$.

> Loop **unrolled**: no loop-control overhead, sequential accesses (better spatial locality and prefetching).

[FIPS 197 §5.1.1, Fig. 2, Table 4; Rijndael AES Implementation §3.1; AESFINAL/utils.c]

Row r is cyclically shifted **left** by r bytes (row 0 unchanged):

$$s'_{r,c} = s_{r,(c+r) \bmod 4}, \qquad 0 \le r, c < 4$$



- In flat indices ($s_{r,c} = \text{state}[r+4c]$) row 1 is a 4-cycle, row 2 swaps $2 \leftrightarrow 10$, $6 \leftrightarrow 14$, row 3 is a 4-cycle the other way.
- Diffusion: the 4 bytes of a column go to 4 columns.

utils.c :: ShiftRows

```
void ShiftRows(u8* state) {
    // Shift second row
    u8 temp = state[1];
    state[1]  = state[5];
    state[5]  = state[9];
    state[9]  = state[13];
    state[13] = temp;

    // Shift third row
    temp = state[2];
    state[2]  = state[10];
    state[10] = temp;
    temp = state[6];
    state[6]  = state[14];
    state[14] = temp;

    // Shift fourth row
    temp = state[15];
    state[15] = state[11];
    state[11] = state[7];
    state[7]  = state[3];
    state[3]  = temp;
}
```

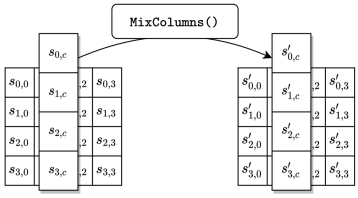
[FIPS 197 §5.1.2, Eq. 5.5, Fig. 3; AESFINAL/utils.c]

MIXCOLUMNS: SPECIFICATION

Every column of the state is multiplied by a fixed matrix over $\mathrm{GF}(2^8)$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \quad 0 \le c < 4$$

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}$$
$$s'_{1,c} = s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c}$$
$$s'_{2,c} = s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c})$$
$$s'_{3,c} = (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c})$$



> Circulant matrix; first column $[\{02\}, \{01\}, \{01\}, \{03\}]$; coefficients are cheap on 8-bit CPUs.

> With ShiftRows it gives **diffusion**: one input byte affects 4 output bytes.

[FIPS 197 §4.3, §5.1.3 (Eq. 5.6-5.8), Fig. 4]

MIXCOLUMNS: EFFICIENT EVALUATION WITH XTIME

For a column $[a_0, a_1, a_2, a_3]$ let $t = a_0 \oplus a_1 \oplus a_2 \oplus a_3$. Then, with indices mod 4,

$$a'_i = a_i \oplus \mathrm{xtime}(a_i \oplus a_{i+1}) \oplus t.$$

Check for $i = 0$ (using $\{03\} \bullet a = \{02\} \bullet a \oplus a$):

$$\begin{aligned} a_0 \oplus \{02\} \bullet (a_0 \oplus a_1) \oplus t &= \{02\} \bullet a_0 \oplus \{02\} \bullet a_1 \oplus a_1 \oplus a_2 \oplus a_3 \\ &= \{02\} \bullet a_0 \oplus \{03\} \bullet a_1 \oplus a_2 \oplus a_3. \end{aligned}$$

- The only field multiplication is xtime.
- Per column: 1 four-way XOR, 4 xtime, 8 XOR.

utils.c :: MixColumns (column 0)

```
void MixColumns(u8* s) {
    // Mix first column
    u8 t = s[0] ^ s[1] ^ s[2] ^ s[3];
    u8 temp = s[0];
    s[0] ^= xtime(s[0] ^ s[1]) ^ t;
    s[1] ^= xtime(s[1] ^ s[2]) ^ t;
    s[2] ^= xtime(s[2] ^ s[3]) ^ t;
    s[3] ^= xtime(s[3] ^ temp) ^ t;
    ...
    // Mix fourth column
    t = s[12] ^ s[13] ^ s[14] ^ s[15];
    temp = s[12];
    s[12] ^= xtime(s[12] ^ s[13]) ^ t;
    ...
    s[15] ^= xtime(s[15] ^ temp)  ^ t;
}
```

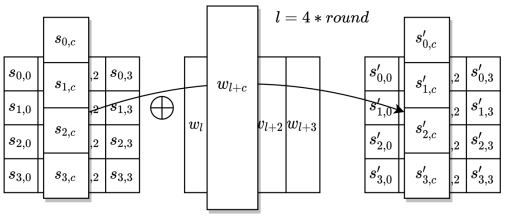
temp keeps the old a_0, needed for a'_3 after a_0 was overwritten.

[FIPS 197 §5.1.3; Rijndael AES Implementation §3.3; AESFINAL/utils.c]

The round key (4 words of the key schedule) is XORed into the 4 columns:

$$[s'_{0,c}, s'_{1,c}, s'_{2,c}, s'_{3,c}] = [s_{0,c}, s_{1,c}, s_{2,c}, s_{3,c}] \oplus w[4 \cdot round + c]$$

for $0 \leq c < 4$ and $0 \leq round \leq N_r$.



> In the figure $l = 4 \cdot round$. Self-inverse: $(s \oplus k) \oplus k = s$.

> One 128-bit XOR handles all 16 bytes: SIMD (SSE2).

utils.c :: AddRoundKey

```
void AddRoundKey(u8* state, u8* ExpandedKey,
                 u8 round) {
    // Load 16 bytes from the state and
    // the corresponding round key
    __m128i* stateVec = (__m128i*)state;
    __m128i* roundKeyVec =
        (__m128i*)(ExpandedKey + 16 * round);

    // XOR 16 bytes at once
    *stateVec = _mm_xor_si128(*stateVec,
                              *roundKeyVec);
}
```

Round key $round$ starts at byte $16 \cdot round$ of ExpandedKey.

[FIPS 197 §5.1.4 (Eq. 5.9), Fig. 5; Rijndael AES Implementation §3.4; AESFINAL/utils.c]

KEYEXPANSION: ALGORITHM

```
1 KeyExpansion(key): w[i] ← key[4i..4i+3], 0 ≤ i < N_k
2 for i = N_k to 4N_r + 3 do
3    temp ← w[i − 1]
4    if i mod N_k = 0 then
5       temp ← SubWord(RotWord(temp)) ⊕ Rcon[i/N_k]
6    else if N_k > 6 and i mod N_k = 4 then
7       temp ← SubWord(temp)
8    w[i] ← w[i − N_k] ⊕ temp
```

> $\mathrm{RotWord}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0]$

> $\mathrm{SubWord}([a_0, \ldots, a_3]) = [\mathrm{SBox}(a_0), \ldots, \mathrm{SBox}(a_3)]$

> $\mathrm{Rcon}[j] = [x^{j-1}, \{00\}, \{00\}, \{00\}]$ in $\mathrm{GF}(2^8)$, i.e. x^{j-1} by repeated xtime.

> Lines 6-7 apply to AES-256 only; AES-128 uses $N_k = 4$.

AES-128, $l = 0, 4, \ldots, 36$:
$w[l+4] = w[l] \oplus \mathrm{SubWord}(\mathrm{RotWord}(w[l+3])) \oplus \mathrm{Rcon}[\frac{l+4}{4}]$, and
$w[l+k] = w[l+k-4] \oplus w[l+k-1]$ for $k = 5, 6, 7$.

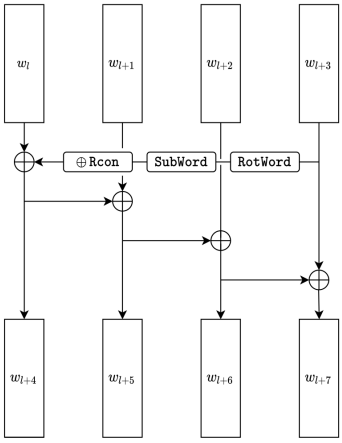
j	$\mathrm{Rcon}[j]_0$	j	$\mathrm{Rcon}[j]_0$
1	01	6	20
2	02	7	40
3	04	8	80
4	08	9	1b
5	10	10	36

utils.c :: Rcon

```
// AES round constants
const u8 Rcon[10] =
{0x01, 0x02, 0x04, 0x08, 0x10,
 0x20, 0x40, 0x80, 0x1B, 0x36};
```

The array stores only the leftmost byte of each $\mathrm{Rcon}[j]$ (the other three are {00}).

[FIPS 197 §5.2, Alg. 2, Table 5; AESFINAL/utils.c]



New word = word 4 earlier ⊕ previous new word; the first also uses RotWord, SubWord, Rcon.

utils.c :: KeyExpansion (AES-128)

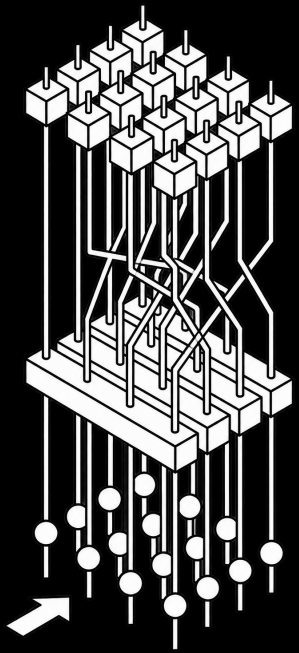
```
void KeyExpansion(u8* key, u8* w) {
    u8 temp[4];
    for (int j = 0; j < 16; ++j)
        w[j] = key[j];
    for (int i = NK; i < 4 * (NR + 1); ++i) {
        temp[0] = w[4*(i-1)];
        temp[1] = w[4*(i-1) + 1];
        temp[2] = w[4*(i-1) + 2];
        temp[3] = w[4*(i-1) + 3];
        if (i % NK == 0) {  // RotWord + SubWord + Rcon
            u8 t = temp[0];
            temp[0] = sbox[temp[1]] ^ Rcon[(i/NK) - 1];
            temp[1] = sbox[temp[2]];
            temp[2] = sbox[temp[3]];
            temp[3] = sbox[t];
        }
        w[4*i]     = w[4*(i-NK)]     ^ temp[0];
        w[4*i + 1] = w[4*(i-NK) + 1] ^ temp[1];
        w[4*i + 2] = w[4*(i-NK) + 2] ^ temp[2];
        w[4*i + 3] = w[4*(i-NK) + 3] ^ temp[3];
    }
}
```

[FIPS 197 Fig. 6; Rijndael AES Implementation §3.5; AESFINAL/utils.c]

> cd section_04

04

DECRYPTION



Invert every transformation and run them in **reverse order**:

1 **procedure** InvCipher(in, N_r, w)
2 $state \leftarrow$ AddRoundKey($in, w[4N_r..4N_r+3]$)
3 **for** $round = N_r - 1$ **downto** 1 **do**
4 InvShiftRows
5 InvSubBytes
6 AddRoundKey($\cdot, w[4\,round..4\,round+3]$)
7 InvMixColumns
8 InvShiftRows; InvSubBytes
9 AddRoundKey($\cdot, w[0..3]$)

AddRoundKey is its own inverse. Decryption keeps the encryption key schedule w and uses the round keys in reverse order.

utils.c :: InvCipher

```
void InvCipher( u8* state, u8* ExpandedKey) {
    AddRoundKey(state, ExpandedKey, 10);

    for (u8 round = NR-1; round > 0; round--) {
        InvShiftRows(state);
        InvSubBytes(state);
        AddRoundKey(state, ExpandedKey, round);
        InvMixColumns(state);
    }

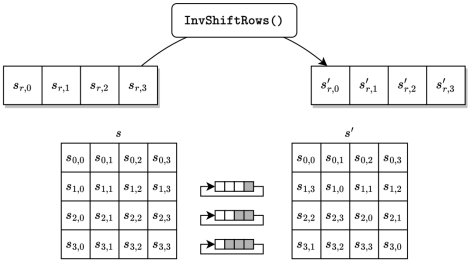
    InvShiftRows(state);
    InvSubBytes(state);
    AddRoundKey(state, ExpandedKey, 0);
}
```

This is the function called by ecb_decrypt and cbc_decrypt.

[FIPS 197 §5.3, Alg. 3; Rijndael AES Implementation §4; AESFINAL/utils.c]

Row r is cyclically shifted **right** by r bytes:

$$s'_{r,c} = s_{r,(c-r) \bmod 4}, \qquad 0 \le r, c < 4$$



- ShiftRows maps column c to $c - r$ and InvShiftRows maps it back: $(c - r) + r \equiv c \pmod 4$.
- Row 2 is self-inverse: shifting by 2 and by -2 coincide mod 4.

utils.c :: InvShiftRows

```
void InvShiftRows(u8* state) {
    // Shift back second row
    u8 temp = state[13];
    state[13] = state[9];
    state[9] = state[5];
    state[5] = state[1];
    state[1] = temp;

    // Shift back third row
    temp = state[2];
    state[2] = state[10];
    state[10] = temp;
    temp = state[6];
    state[6] = state[14];
    state[14] = temp;

    // Shift back fourth row
    temp = state[3];
    state[3] = state[7];
    state[7] = state[11];
    state[11] = state[15];
    state[15] = temp;
}
```

[FIPS 197 §5.3.1, Eq. 5.12, Fig. 9; AESFINAL/utils.c]

InvSBox is SBox with the roles of input and output swapped:

$$\mathrm{InvSBox}(\mathrm{SBox}(b)) = b, \qquad s'_{r,c} = \mathrm{InvSBox}(s_{r,c})$$

- Construction: $\mathrm{InvSBox} = g \circ f^{-1}$ (inverse affine map, then inversion in $\mathrm{GF}(2^8)$).
- Example: $\mathrm{SBox}(\{53\}) = \{ed\}$ and $\mathrm{InvSBox}(\{ed\}) = \{53\}$.
- Implemented as a second 256-byte table, again with the loop unrolled.

utils.c :: InvSbox, InvSubBytes

```
const u8 InvSbox[256]  = {
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36,
    0xa5, 0x38, 0xbf, 0x40, 0xa3, 0x9e,
    ...
};

void InvSubBytes(u8* state) {
    state[0]  = InvSbox[state[0]];
    state[1]  = InvSbox[state[1]];
    ...
    state[15] = InvSbox[state[15]];
}
```

[FIPS 197 §5.3.2, Table 6; Rijndael AES Implementation §2.2; AESFINAL/utils.c]

INVMIXCOLUMNS: BARRETO'S FACTORISATION

$$\underbrace{\begin{bmatrix} \texttt{0e} & \texttt{0b} & \texttt{0d} & \texttt{09} \\ \texttt{09} & \texttt{0e} & \texttt{0b} & \texttt{0d} \\ \texttt{0d} & \texttt{09} & \texttt{0e} & \texttt{0b} \\ \texttt{0b} & \texttt{0d} & \texttt{09} & \texttt{0e} \end{bmatrix}}_{\texttt{InvMixColumns}} = \underbrace{\begin{bmatrix} \texttt{02} & \texttt{03} & \texttt{01} & \texttt{01} \\ \texttt{01} & \texttt{02} & \texttt{03} & \texttt{01} \\ \texttt{01} & \texttt{01} & \texttt{02} & \texttt{03} \\ \texttt{03} & \texttt{01} & \texttt{01} & \texttt{02} \end{bmatrix}}_{\texttt{MixColumns}} \underbrace{\begin{bmatrix} \texttt{05} & \texttt{00} & \texttt{04} & \texttt{00} \\ \texttt{00} & \texttt{05} & \texttt{00} & \texttt{04} \\ \texttt{04} & \texttt{00} & \texttt{05} & \texttt{00} \\ \texttt{00} & \texttt{04} & \texttt{00} & \texttt{05} \end{bmatrix}}_{P}$$

- As polynomials:
  $d(x) = \big(\{\texttt{04}\}x^2 + \{\texttt{05}\}\big) \otimes c(x) \bmod (x^4 + \{\texttt{01}\})$.
- Direct coefficients $\{\texttt{09}\}, \{\texttt{0b}\}, \{\texttt{0d}\}, \{\texttt{0e}\}$ need many xtime calls.
- Instead: a cheap **pre-processing** P, then the existing MixColumns:

$$u = \text{xtime}^2(a_0 \oplus a_2), \quad v = \text{xtime}^2(a_1 \oplus a_3)$$

$$(a_0, a_1, a_2, a_3) \leftarrow (a_0 \oplus u,\ \ a_1 \oplus v,\ \ a_2 \oplus u,\ \ a_3 \oplus v)$$

utils.c :: InvMixColumns

```
void InvMixColumns(u8* s) {
    u8 u = xtime(xtime(s[0] ^ s[2]));
    u8 v = xtime(xtime(s[1] ^ s[3]));
    s[0] ^= u;
    s[1] ^= v;
    s[2] ^= u;
    s[3] ^= v;
    ...  // same for columns 1, 2, 3
    MixColumns(s);
}
```

[FIPS 197 §5.3.3 (Eq. 5.13-5.15); Rijndael AES Implementation §4, Eq. (1) (P. Barreto); AESFINAL/utils.c]

EQINVCIPHER AND KEYEXPANSIONEIC

Same *order* as Cipher, each step replaced by its inverse:

1 AddRoundKey($in, dw[4N_r..4N_r+3]$)
2 **for** $round = N_r - 1$ **downto** 1 **do**
3 InvSubBytes; InvShiftRows
4 InvMixColumns
5 AddRoundKey($\cdot, dw[4\,round.. + 3]$)
6 InvSubBytes; InvShiftRows
7 AddRoundKey($\cdot, dw[0..3]$)

Key schedule dw: $dw[i] = w[i]$, except for $1 \le round \le N_r - 1$:

$$dw[4round..+3] \leftarrow \text{InvMixColumns}\big(w[4round..+3]\big)$$

Why equivalent: InvSubBytes and InvShiftRows commute, and InvMixColumns is linear, so $\text{InvMix}(s \oplus k) = \text{InvMix}(s) \oplus \text{InvMix}(k)$.

utils.c :: EqInvCipher

```
void EqInvCipher(u8* state, u8* ExpandedKey) {
    AddRoundKey(state, ExpandedKey, 10);
    for (u8 round = NR-1; round > 0; round--) {
        InvSubBytes(state);
        InvShiftRows(state);
        InvMixColumns(state);
        AddRoundKey(state, ExpandedKey, round);
    }
    InvSubBytes(state);
    InvShiftRows(state);
    AddRoundKey(state, ExpandedKey, 0);
}
```

utils.c :: KeyExpansionEIC (tail)

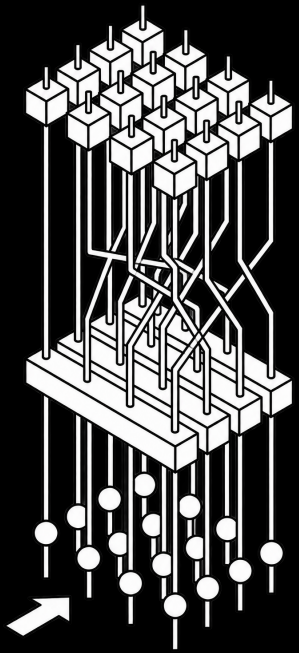
```
    // Apply InvMixColumns to the key schedule
    for (int round = 1; round < NR; ++round) {
        int i = 16 * round;
        InvMixColumns(dw+i);
    }
```

[FIPS 197 §5.3.5, Alg. 4-5; Rijndael AES Implementation §4; AESFINAL/utils.c]

> cd section_05

05

LOOK-UP TABLES



FROM ROUND TO T-TABLES (1/2)

Let a be the round input, b the output of SubBytes, c of ShiftRows, d of MixColumns ($0 \le j < N_b$, shift offsets C_i, indices mod N_b):

$$b_{i,j} = \mathrm{SBox}[a_{i,j}], \qquad c_{i,j} = b_{i,\,j+C_i}, \qquad \begin{bmatrix} d_{0,j} \\ d_{1,j} \\ d_{2,j} \\ d_{3,j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} c_{0,j} \\ c_{1,j} \\ c_{2,j} \\ c_{3,j} \end{bmatrix}$$

Substitute and read the matrix product as a combination of its **four columns**:

$$\begin{bmatrix} d_{0,j} \\ d_{1,j} \\ d_{2,j} \\ d_{3,j} \end{bmatrix} = \begin{bmatrix} 02 \\ 01 \\ 01 \\ 03 \end{bmatrix} \bullet \mathrm{SBox}[a_{0,j+C_0}] \;\oplus\; \begin{bmatrix} 03 \\ 02 \\ 01 \\ 01 \end{bmatrix} \bullet \mathrm{SBox}[a_{1,j+C_1}] \;\oplus\; \begin{bmatrix} 01 \\ 03 \\ 02 \\ 01 \end{bmatrix} \bullet \mathrm{SBox}[a_{2,j+C_2}] \;\oplus\; \begin{bmatrix} 01 \\ 01 \\ 03 \\ 02 \end{bmatrix} \bullet \mathrm{SBox}[a_{3,j+C_3}]$$

The input to each term is a *single byte*, so each term can be pre-computed for all 256 values.

[Rijndael AES Implementation §5.1, Eq. (1)-(5)]

FROM ROUND TO T-TABLES (2/2)

Four tables of 256 four-byte words ($\mathrm{S} = \mathrm{SBox}$):

$$T_0[a] = [02 \bullet \mathrm{S}[a],\ \mathrm{S}[a],\ \mathrm{S}[a],\ 03 \bullet \mathrm{S}[a]]^T$$
$$T_1[a] = [03 \bullet \mathrm{S}[a],\ 02 \bullet \mathrm{S}[a],\ \mathrm{S}[a],\ \mathrm{S}[a]]^T$$
$$T_2[a] = [\mathrm{S}[a],\ 03 \bullet \mathrm{S}[a],\ 02 \bullet \mathrm{S}[a],\ \mathrm{S}[a]]^T$$
$$T_3[a] = [\mathrm{S}[a],\ \mathrm{S}[a],\ 03 \bullet \mathrm{S}[a],\ 02 \bullet \mathrm{S}[a]]^T$$

One round, column j, *including* AddRoundKey:

$$[d_{0,j}, \ldots, d_{3,j}]^T = T_0[a_{0,j+C_0}] \oplus T_1[a_{1,j+C_1}] \oplus T_2[a_{2,j+C_2}] \oplus T_3[a_{3,j+C_3}] \oplus w[4\,round + j]$$

cost and trade-offs

- 4 look-ups + 4 XORs per column per round, for 32-bit or wider words.
- $4 \times 256 \times 4$ bytes $=$ **4 kB** of tables (T_k are byte-rotations of T_0).
- The final round has no MixColumns and uses plain SBox.
- Secret-dependent table indices make **cache-timing attacks** possible (Osvik-Shamir-Tromer).

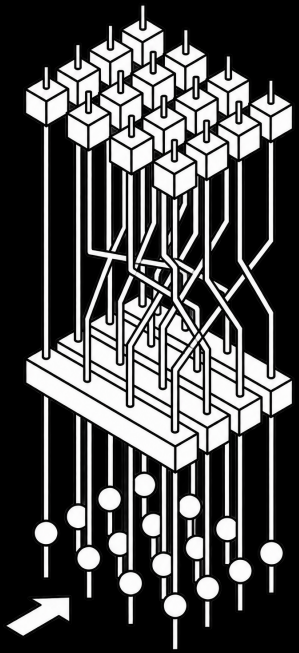
AESFINAL does *not* use T-tables: it keeps the compact SubBytes/MixColumns code with a 256-byte S-box.

[Rijndael AES Implementation §5.1; Osvik, Shamir, Tromer, J. Cryptology 23 (2010)]

> cd section_06

06

MODES OF OPERATION



MODES OF OPERATION: IV AND PADDING

A message $m = m_0 m_1 \ldots$ is processed in 128-bit blocks; the mode defines how E_K is used (NIST SP 800-38A).

- **IV**: a block-sized value for CBC, OFB (and CFB); a nonce for CTR. We draw it with OpenSSL RAND_bytes and store it as the first 16 bytes of the file.
- ECB and CBC need whole blocks: **PKCS#7 padding**. If the last block has ℓ bytes ($0 \le \ell < 16$), append $p = 16 - \ell$ bytes each of value p (so $1 \le p \le 16$; a full block of padding is added when $\ell = 0$).
- Decryption reads the last byte p, checks that the last p bytes equal p, and removes them.

Modes covered: ECB, CBC, OFB, CTR (CFB is implemented in the code but not presented).

moops.c :: IV and padding

```
unsigned char IV[16];

void generate_iv() {
    if (!RAND_bytes(IV, sizeof(IV))) {
        perror("Error generating random IV");
        exit(EXIT_FAILURE);
    }
}
...
u8 pkcs7_unpad(u8* plaintext) {
    u8 pad_value = plaintext[15];
    if (pad_value > 0 && pad_value <= 16) {
        for (u8 i = 0; i < pad_value; ++i) {
            if (plaintext[16 - 1 - i] != pad_value
 ↪ ) {
                ...
            }
        }
        return pad_value;
    } else {
        ...
    }
}
```

[Rijndael AES Implementation §6; NIST SP 800-38A; Kaliski, RFC 2315; AESFINAL/moops.c]

ECB: ELECTRONIC CODEBOOK

Encryption: $c_i = E_K(m_i)$
Decryption: $m_i = D_K(c_i)$

Weakness: $m_i = m_j \Rightarrow c_i = c_j$. Patterns of a low-entropy plaintext space survive encryption; ECB is virtually never used in practice.

moops.c :: ecb_encrypt

```
void ecb_encrypt(FILE* ifile, FILE *ofile, u8*
  ↪ ExpandedKey){
    u8 in_block[16];
    u8 bytes_read;
    while ((bytes_read = fread(in_block, 1, 16, ifile))
  ↪  == 16) {
        Cipher(in_block, ExpandedKey);
        fwrite(in_block, 1, 16, ofile);
    }
    // Apply PKCS#7 padding
    u8 padding_value = 16 - bytes_read;
    for (u8 i = bytes_read; i < 16; ++i)
        in_block[i] = padding_value;
    Cipher(in_block, ExpandedKey);
    fwrite(in_block, 1, 16, ofile);
}
```

moops.c :: ecb_decrypt

```
void ecb_decrypt(FILE* ifile, FILE *ofile, u8*
  ↪ ExpandedKey){
    u8 in_block[16], out_block[16];
    u8 bytes_read;
    while (fread(in_block, 1, 16, ifile) > 0) {
        InvCipher(in_block, ExpandedKey);
        fwrite(in_block, 1, 16, ofile);
    }
    // Handle the last block with padding removal
    u8 pad_len = pkcs7_unpad(in_block);
    fseek(ofile, -pad_len,SEEK_END);
    long current_pos = ftell(ofile);
    ftruncate(fileno(ofile), current_pos);
}
```

[Rijndael AES Implementation §6.1; AESFINAL/moops.c]

CBC: CIPHER BLOCK CHAINING

Encryption: $\begin{cases} c_0 = E_K(m_0 \oplus IV) \\ c_i = E_K(m_i \oplus c_{i-1}), \ i \geq 1 \end{cases}$ **Decryption:** $\begin{cases} m_0 = D_K(c_0) \oplus IV \\ m_i = D_K(c_i) \oplus c_{i-1}, \ i \geq 1 \end{cases}$

Random IV $\Rightarrow$ CPA-secure (CCA needs authenticated encryption). Encryption is sequential; decryption is parallelisable (m_i needs only c_i, c_{i-1}).

moops.c :: cbc_encrypt

```
void cbc_encrypt(FILE* ifile, FILE *ofile, u8*
  ↪ ExpandedKey){
    u8 plaintext[16];
    u8 bytes_read;
    generate_iv();
    fwrite(IV, 1, 16, ofile);
    while ((bytes_read = fread(plaintext, 1, 16, ifile)
  ↪ ) == 16) {
        for (u8 i = 0; i < 16; ++i)
            IV[i] ^= plaintext[i] ;
        Cipher(IV, ExpandedKey);
        fwrite(IV, 1, 16, ofile);
    }
    ... // last block: PKCS#7 pad, XOR with IV,
    ... // Cipher, write (same chaining)
}
```

moops.c :: cbc_decrypt

```
void cbc_decrypt(FILE* ifile, FILE *ofile, u8*
  ↪ ExpandedKey){
    u8 input[16], temp[16], output[16];
    u8 bytes_read;
    fread(IV, 1, 16, ifile);
    while (fread(input, 1, 16, ifile) > 0) {
        for (u8 i = 0; i < 16; i++)
            temp[i] = input[i];
        InvCipher(input, ExpandedKey);
        for (u8 i = 0; i < 16; ++i)
            input[i] ^= IV[i];
        for (u8 i = 0; i < 16; i++)
            IV[i] = temp[i];
        fwrite(input, 1, 16, ofile);
    }
    ... // unpad last block, truncate file
}
```

[Rijndael AES Implementation §6.2; AESFINAL/moops.c. Global IV is the chaining register.]

OFB: OUTPUT FEEDBACK

Synchronous stream cipher: the keystream is the iterated encryption of the IV.

$$\begin{cases} z_0 = IV \\ z_i = E_K(z_{i-1}), \ \ i \geq 1 \\ c_i = z_i \oplus m_i \end{cases}$$

Decryption is identical: $m_i = z_i \oplus c_i$.

- E_K is used for **both** directions; D_K is never needed.
- No padding: the last partial block is simply truncated.
- Keystream independent of the data (can be pre-computed), but strictly sequential.
- No security against CCA (Rogaway); never reuse (K, IV).

moops.c :: ofb (encrypt and decrypt)

```
void ofb(FILE* ifile, FILE *ofile, u8* ExpandedKey,
         char choice){
    u8 in_block[16];
    u8 bytes_read;
    generate_iv();
    if(choice == 'e') fwrite(IV, 1, 16, ofile);
    else fread(IV, 1, 16, ifile);

    while ((bytes_read = fread(in_block, 1, 16, ifile)
  ↪ ) > 0) {
        Cipher(IV, ExpandedKey);
        xor(in_block,IV);
        fwrite(in_block, 1, bytes_read, ofile);
    }
}
```

Cipher(IV,…) turns z_{i-1} into z_i in place; xor gives $c_i = z_i \oplus m_i$. choice selects write or read of the IV.

[Rijndael AES Implementation §6.4; AESFINAL/moops.c]

CTR: COUNTER MODE

Keystream from encrypting successive counter values ($\ell = 128$):

$$\begin{cases} T_i = (ctr + i - 1) \bmod 2^\ell \\ c_i = E_K(T_i) \oplus m_i, \;\; i \geq 1 \end{cases}$$

Decryption: $m_i = E_K(T_i) \oplus c_i$.

- ctr = 96-bit non-repeating nonce $\|$ 32-bit block counter, starting at 0.
- Blocks are independent: **parallel** encryption and decryption, pre-processing, random access.
- Software and hardware efficient; marked as the best choice among the modes in Rijndael AES Implementation.
- Never reuse a (K, nonce) pair.

moops.c :: ctr (encrypt and decrypt)

```
void ctr(FILE *ifile, FILE *ofile, u8* ExpandedKey,
         char choice){
    generate_iv();
    if(choice == 'e') fwrite(IV, 1, 16, ofile);
    else fread(IV, 1, 16, ifile);
    IV[0]=IV[1]=IV[2]=IV[3]=0x00;
    u8 input[16], in_block[16];
    u8 bytes_read;
    while ((bytes_read = fread(in_block, 1, 16, ifile)
 ↪) > 0) {
        for (u8 i = 0; i < 16; i++)
            input[i]=IV[i];
        Cipher(input,ExpandedKey);
        xor(in_block, input);
        fwrite(in_block, 1, bytes_read, ofile);
        for (u8 i = 0; i < 4; ++i)
            if (++IV[i] != 0) break;
    }
}
```

[Rijndael AES Implementation §6.5; AESFINAL/moops.c (xor operands swapped vs. original: see notes); counter in IV[0..3]]

MODES: COMPARISON

mode	uses	padding	IV / nonce	par. enc.	par. dec.	keystream pre-computable
ECB	E_K / D_K	yes	none	yes	yes	-
CBC	E_K / D_K	yes	random IV	no	yes	-
OFB	E_K only	no	unique IV	no	no	yes
CTR	E_K only	no	unique nonce	yes	yes	yes

> ECB leaks equality of blocks; CBC/OFB/CTR hide it when the IV is fresh.

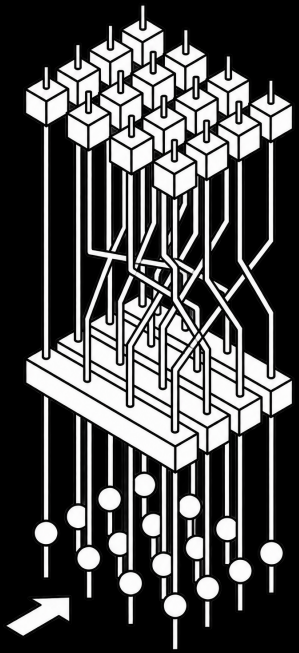
> None of the confidentiality modes authenticates the data: use authenticated encryption against active attackers.

[Rijndael AES Implementation §6; NIST SP 800-38A]

> cd section_07

07

IMPLEMENTATION



OUR IMPLEMENTATION: DRIVER AND USAGE

AESFINAL/ layout

```
AES.c      command-line driver
utils.c/h  S-box, round steps,
           key expansion, Cipher,
           InvCipher
moops.c/h  ECB, CBC, CFB, OFB, CTR;
           IV; PKCS#7
```

build and run (readme.txt)

```
gcc -c utils.c
gcc -c moops.c
gcc -o AES AES.c utils.o moops.o -lcrypto
./AES e 1 test.pdf out.pdf KEY16CHARS...
  <op> <mode> <in> <out> <key>
```

op: e/d. mode: 1 ECB, 2 CBC, 3 CFB, 4 OFB, 5 CTR.

Key: 16 chars.

AES.c :: main (excerpt)

```
int main(int argc, char* argv[]) {
    if (argc != 6) {
        print_usage(argv[0]);
        return 1;
    }
    ...
    u8 key[16];
    for (int i = 0; i < 16; i++)
        sscanf(&argv[5][i], "%c", &key[i]);
    ...
    u8 ExpandedKey[176];
    KeyExpansion(key, ExpandedKey);

    FILE *ifile = fopen(argv[3], "rb");
    FILE *ofile = fopen(argv[4], "wb");
    ...
    if (argv[1][0] == 'e') {
        switch (argv[2][0]) {
        case '1':
            ecb_encrypt(ifile, ofile, ExpandedKey);
            break;
        ...
        default:
            ctr(ifile, ofile, ExpandedKey, 'e');
```

[AESFINAL/AES.c, readme.txt]

[1] NIST. *Advanced Encryption Standard (AES)*, FIPS PUB 197-upd1, 26 Nov 2001 (updated 9 May 2023). doi:10.6028/NIST.FIPS.197-upd1. [AES_Docs.pdf]

[2] *Rijndael AES Implementation.*

[3] *Advanced Encryption Standard*, Wikipedia. en.wikipedia.org/wiki/Advanced_Encryption_Standard (history slides).

[4] *Advanced Encryption Standard process*, Wikipedia. en.wikipedia.org/wiki/Advanced_Encryption_Standard_process (competition timeline, finalists).

[5] J. Daemen, V. Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard.* Springer, 2002 (2nd ed. 2020).

[6] NIST. *Recommendation for Block Cipher Modes of Operation*, SP 800-38A, 2001. doi:10.6028/NIST.SP.800-38A.

[7] B. Kaliski. *PKCS #7: Cryptographic Message Syntax*, RFC 2315, 1998.

[8] D. A. Osvik, A. Shamir, E. Tromer. Efficient Cache Attacks on AES, and Countermeasures. *J. Cryptology* 23 (2010), 37-71.

[9] P. Rogaway. *Evaluation of Some Blockcipher Modes of Operation*, 2011.

[10] S. Gueron (Intel). *Intel Advanced Encryption Standard (AES) New Instructions Set*, white paper, May 2010.

[11] J. Nechvatal et al. *Report on the Development of the AES*, J. Res. NIST 106 (2001). doi:10.6028/jres.106.023.

> exit 0

Thank you

Questions?

> git clone



https://github.com/0r-gh0/AES-Presentation

slides, LaTeX source and C code